Assignments-3

Aim

*To implement the A Search algorithm to solve the 8-Puzzle problem using an appropriate heuristic function.**

Objectives

1. Understand the A* Search algorithm**.**
2. Implement A* Search for the 8-Puzzle problem.
3. Use Manhattan Distance as a heuristic function.
4. Understand the evaluation function **f(n) = g(n) + h(n)**.
5. Find an optimal path from the initial state to the goal state.

Problem Statement

The **8-Puzzle** consists of 8 numbered tiles and one blank space arranged in a **3 × 3 grid**.

The blank space can move **up, down, left, or right**.

The objective is to transform the given **initial state** into the **goal state** using the minimum number of moves.

Initial State

```
1  2  3
4  0  6
7  5  8
```

Here, `0` represents the blank space.

Goal State

```
1  2  3
4  5  6
7  8  0
```

A* Evaluation Function

A* uses:

f(n)=g(n)+h(n)f(n) = g(n) + h(n)

Where:

- **g(n)** = Cost from the initial state to the current state.
- **h(n)** = Estimated cost from the current state to the goal.
- **f(n)** = Total estimated cost.

Heuristic Used: Manhattan Distance

For each tile:

h(n)=|x1−x2|+|y1−y2|h(n) = |x_1-x_2| + |y_1-y_2|

The Manhattan distances of all misplaced tiles are added together.

Algorithm

1. Store the initial state.
2. Store the goal state.
3. Calculate the heuristic value using Manhattan Distance.
4. Insert the initial state into the priority queue.
5. Select the state having the lowest `f(n)`.
6. Generate all possible successor states.
7. Calculate `g(n)`, `h(n)` and `f(n)` for each successor.
8. Add unexplored states to the priority queue.
9. Repeat until the goal state is reached.
10. Display the sequence of moves.

Program Code:

```
# AI Practical 3

# A* Search Algorithm for 8-Puzzle Problem


import heapq



# Goal state
```

```
goal = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)



# Initial state
initial = (1, 2, 3,
           4, 0, 6,
           7, 5, 8)



# Manhattan Distance Heuristic
def heuristic(state):
    distance = 0

    for i in range(9):

        if state[i] == 0:
            continue

        current_row = i // 3
        current_col = i % 3
```

```
        goal_index = goal.index(state[i])

        goal_row = goal_index // 3
        goal_col = goal_index % 3

        distance += abs(current_row - goal_row)
        distance += abs(current_col - goal_col)

    return distance


# Generate successor states
def get_neighbors(state):

    neighbors = []

    blank = state.index(0)

    row = blank // 3
    col = blank % 3

    moves = [
        (-1, 0, "Up"),
```

```
    (1, 0, "Down"),
    (0, -1, "Left"),
    (0, 1, "Right")
]

for dr, dc, move in moves:

    new_row = row + dr
    new_col = col + dc

    if 0 <= new_row < 3 and 0 <= new_col < 3:

        new_blank = new_row * 3 + new_col

        new_state = list(state)

        new_state[blank], new_state[new_blank] = \
            new_state[new_blank], new_state[blank]

        neighbors.append((tuple(new_state), move))

return neighbors
```

```
# A* Search
def a_star(initial):

    priority_queue = []

    g_cost = {initial: 0}

    parent = {initial: None}

    move_taken = {initial: None}

    h = heuristic(initial)

    heapq.heappush(
        priority_queue,
        (h, 0, initial)
    )

    visited = set()

    while priority_queue:
```

```
f, g, current = heapq.heappop(priority_queue)

if current in visited:
    continue

visited.add(current)

# Goal test
if current == goal:

    path = []

    while current is not None:
        path.append(
            (current, move_taken[current])
        )
        current = parent[current]

    path.reverse()

    return path

# Generate successors
```

```
    for neighbor, move in get_neighbors(current):

        new_g = g + 1

        if neighbor not in g_cost or new_g < g_cost[neighbor]:

            g_cost[neighbor] = new_g

            h = heuristic(neighbor)

            f = new_g + h

            parent[neighbor] = current

            move_taken[neighbor] = move

            heapq.heappush(
                priority_queue,
                (f, new_g, neighbor)
            )

return None
```

```
# Display solution
solution = a_star(initial)

if solution:

    print("Solution Found!")
    print("Number of moves:", len(solution) - 1)
    print()

    for i, (state, move) in enumerate(solution):

        print("Step:", i)

        if move:
            print("Move:", move)

        for j in range(0, 9, 3):
            print(state[j:j+3])

        print()

else:
```

```
    print("No solution found.")
```

conclusion:

Thus, the A* Search algorithm was successfully implemented for the 8-Puzzle problem using Manhattan Distance as the heuristic function, and the optimal sequence of moves from the initial state to the goal state was obtained.

Out put:

```
Solution Found!
Number of moves: 2

Step: 0
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

Step: 1
Move: Down
(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

Step: 2
Move: Right
(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
```